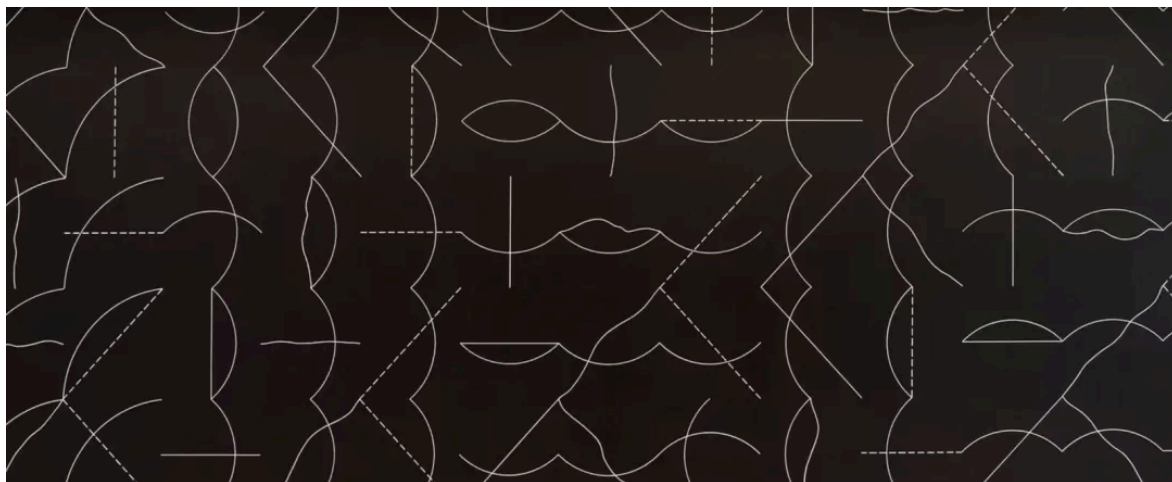


重新思考研发基础设施：当 Agent 成为第一公民

原创 许晓斌 阿里技术 2026年6月3日 18:09 浙江



这是2026年的第20篇文章

本文阅读时间：约30分钟

(本文作者晓斌，阿里巴巴研发基础设施负责人)

前言

从一个周报系统说起

过去几个月，我做了一个小系统：收集团队在多个异构系统（代码托管、项目管理、数据平台、IM）中的活动数据，聚合分析后生成一份周报，发布到内部 Pages 站点。功能不复杂，但这个系统的构成方式让我想了很多。

它的架构可以用一句话概括：connector 打通异构系统 → 规范化成统一活动流 → 用一个意图 (skill) 驱动聚合与排版 → 生成单文件 HTML → 推送到 Pages。（这里的 connector 是我自己写的一组小工具，用 CLI 的方式让 agent 连接各种基础设施——背后的核心是各平台的 CLI。）

这里面有三类组成部分。第一类是 **agent**——它是核心驱动力，读取一份叫 `SKILL.md` 的意图描述，连接各个数据源，在运行时动态生成 Python 脚本做数据处理，动态生成 HTML 做渲染。第二类是**代码**——有一些 Python 和 Shell 脚本，但关键区别在于，大部分代码是 agent 每次运行时按需生成的，生命周期很短。第三类是 **infra**——内部 Pages 做托管，CI 做部署，Git 做版本控制——但对我来说，这些几乎“不可见”了。Agent 直接 push 并触发部署，我不需要关心 CI 配置或部署流程。

这个系统有一个很有意思的特性：它的意图是一个活物，不是冻结的需求。`SKILL.md` 用自然语言描述“要产出什么报告”，仅在一轮迭代里，这个意图就改了很多次——加会议决策模块、加交叉洞察、调整模块顺序、改标题、overview 加关键贡献者。每一次都是改 skill，而不是改一段写死的程序。改完之后 agent 立即重新生成代码并部署，整个循环是分钟级的。

从这个小系统里，我观察到两件事。

- **第一，代码是 generated on the fly 的。** 每次运行时根据当前意图和数据动态生成、执行、丢弃，生命周期极短。代码的生命周期从"月/年"缩短到了"分钟"。你可以把它类比为 JIT 编译——编译器每次运行时生成的机器码不会提交到 Git，因为那是中间产物，不是资产。这个系统里的 Python 脚本和 HTML 也是同样的角色。
- **第二，Infra 变得"不可见"了。** Git、CI、Pages 仍然在工作，但它们退到了后台。我感知到的只是"告诉 agent 我要什么 → 拿到结果"，中间的构建、部署、托管过程是透明的。

这个小系统虽然简单，但它的构成方式让我开始重新想一个更大的问题：如果越来越多的系统以这种方式运行，那我们今天围绕"人写持久化代码"设计的整套 developer infra，还能支撑吗？

01

一个更本质的观察：

软件一直是"意图驱动 + 代码沉淀"的进化体

从周报系统抽象出来，容易得出一个判断："未来的软件系统包含智能的部分和静态的部分"。这个判断不算错，但它是一个静态的描述，没有解释力。如果加入时间维度，会发现一个更本质的东西。

无论是传统研发还是 agent 研发，软件系统一直都是由"意图（不确定性）驱动 + 代码（确定性）沉淀"的进化体。 这个模式从未改变，改变的只是驱动和沉淀的速度与机制。

传统研发中的同一个模式

传统软件研发的循环大家都很熟悉：用户有意图（不确定的、模糊的、变化的）→ 产品经理理解并压缩意图 → 研发把意图翻译成代码（确定性沉淀）→ 代码上线运行 → 新的意图涌入 → 下一个循环。

在这个模式里，**人是意图到代码之间的桥梁**。产品经理做意图压缩，研发做代码翻译。桥梁的带宽决定了循环的速度——通常是周或月级别。这是人的认知带宽的物理约束：一个工程师同时能 hold 住的上文是有限的，一个产品经理同时能推进的需求也是有限的。

Agent 研发中的同一个模式

Agent 参与后，循环变成了：意图（`SKILL.md` / 自然语言描述）→ agent 理解意图并生成代码 → 代码执行 → 结果反馈 → 意图修正 → 下一个循环。

模式没变，**但桥梁从"人"变成了"agent"**，循环速度从周/月压缩到了分钟级。回到周报系统的例子——我在一轮迭代里就改了多次意图，加模块、调顺序、改标题，每次修改后 agent 立即重新生成代码并部署。如果这些改动走传统的产品经理 → 研发 → CR → CI 管线，每一次可能就是一个半天的迭代周期，全部做完可能需要两周。

用这个视角重新看两个案例

我再看另一个案例：镜像仓库建站运维。这是一个跨 6+ 个异构平台、涉及 12+ 个配置文件的结构化 SOP——为新地域搭建镜像仓库托管链路。传统排期 2 人日，AI 辅助后 1-1.5 小时完成。

用"意图驱动 + 代码沉淀"的框架看这两个案例，它们是同一个模式在不同循环速度下的表现：

镜像仓库建站（Agent $\approx$ 50%）的桥梁是 agent + 人。Agent 执行全部技术操作——API 调用创建仓库和配置同步策略、生成十多个配置文件、提交 CR 并推送。人处理风控环节——生产环境的配置推送和白名单变更——这些操作技术上完全可以自动化，但因为错推可能导致线上故障且缺乏灰度回滚能力，所以留给人做。意图 $\rightarrow$ 执行的循环周期从 2 人日压缩到 1.5 小时，但人仍然在循环里——人机边界由出错代价决定，由风控机制决定。

周报系统（Agent $\approx$ 90%）的桥梁几乎完全是 agent。意图 $\rightarrow$ 执行的循环周期是分钟级。人只负责表达意图和验收结果。这里有一个值得注意的细节：周报系统的 connector 不仅连接系统 API，还隐含了一套工作模式规范——`cli-meeting-minutes` connector 指向内部 IM 某个文件夹，只有当人真的把会议纪要放进那个文件夹，这条 connector 才有数据。每加一个 connector，等于同时给团队定义了一种工作模式。这是意图驱动的一个有趣副产品：系统对人的要求从"按流程执行"变成了"按规范沉淀"。

两者的差异不在模式，在速度。

这个 insight 的三个推论

- **推论一：Agent 不是革命，是加速。** 它没有改变软件进化的基本结构，只是把"意图 $\rightarrow$ 代码"的循环速度提高了几个数量级。这个判断很重要，因为它意味着我们不需要推翻已有的认知框架来理解 agent，只需要追问：当速度变化之后，原来在慢速下成立的假设是否还成立？
- **推论二：静态沉淀不会消失。** 不确定性被消除后，确定性的东西就应该用确定性的方式表达——这就是代码存在的理由。用一个巨大的概率模型去逼近一个已经确定的函数，在信息论意义上是荒谬的浪费。所以 agent 系统在成熟后会自然地把验证过的逻辑固化为代码，agent 占比会下降——直到新的意图涌入，agent 占比又上升。

如果把 agent 占比画成时间曲线，应该呈**锯齿形**：意图涌入时陡升（不确定性高，agent 探索），逻辑固化时缓降（不确定性消除，代码沉淀）。两个案例分别代表了两条典型路径：

周报系统是上面那条线——agent-native 系统从接近 100% 的高位出发，连 connector 怎么写、数据怎么聚合都是 agent 在摸索。几轮迭代之后，`connectors.yaml` 稳定了、CI 配置固化了、

`check_connectors.sh` 沉淀了，agent 占比随之下降。然后我说“加一个交叉洞察模块”，新意图涌入，占比又上升——但每次上升的幅度在递减，因为系统的骨架越来越稳固。

镜像服务系统是下面那条线——存量系统从低位出发，研发逐步让 agent 接管越来越多的工作，每次接管都是一个小锯齿：agent 根据新地域参数生成全部配置（上升），配置被审批入库固化（下降），但整体趋势在爬升。

两条线最终都在震荡中趋向各自的均衡点。如果某条线是一条平线，说明有问题——要么需求真的在持续变化（健康），要么该做的代码固化没做（不健康）。锯齿形本身就是一个系统健康度的诊断工具。

- **推论三：模式没变，但模式对基础设施的要求彻底变了。** 当循环速度从月级到分钟级，原来为慢循环设计的基础设施就会成为瓶颈或变得不适用。Git 假设每次变更都值得 commit；CI/CD 假设构建和部署是离散事件；测试环境假设代码是稳定的、可以提前准备好环境来跑；发布系统假设发布是低频的、需要人工审批的；Code Review 假设有人来看每一行代码。这些假设在慢循环下全部成立，在快循环下全部失效。

02

统一框架下的三个变量

上一节建立了“意图驱动 + 代码沉淀”的统一框架。Agent 没有改变这个框架的结构，但改变了框架里三个关键变量的取值。理解这三个变量的变化，就理解了从传统研发到 agent 研发的完整差异。

桥梁带宽：从人到 agent

“意图 → 代码”的桥梁从人换成了 agent，带宽提升了几个数量级。人的角色从“桥梁”后退为“意图的表达者和”结果的验收者”。

镜像仓库建站案例最直接地体现了这一点。传统排期 2 人日，操作本身的难度不高，但跨 6+ 个平台、12+ 个配置文件的编排操作对人的认知带宽要求极高。一个不熟悉业务的工程师，即便拿着 SOP 也很容易出错：不同环境的密钥加密实例不同，密文不可互换；不同 region 之间的镜像同步走不同的流水线；缓存要刷新才能生效。这些细节散落在 SOP 的各个角落，人需要全程保持高度注意力。

Agent 的优势恰恰在于此：不丢上下文（12+ 个配置文件不混）、不漏步骤（6+ 个系统全部覆盖）、不混配置（不同环境严格区分）。这跟“聪明”无关——Standard 级别的模型就够了——这是认知带宽上的碾压。结果是：原来要 2 人日、依赖老手经验的操作，现在是“新手 + AI = 老手的产出”。SOP 从“面向人的菜谱”变成了“面向 agent 的可执行意图”。

在周报系统里，这个变化更彻底。我不需要写代码来实现“加一个会议决策模块”——我只需要在 `SKILL.md` 里表达这个意图，agent 自己生成 Python 脚本去解析会议纪要、提取决策项、格式化成 HTML。如果结果不对，我调整意图描述，agent 重新生成。整个过程中，我的角色不是工程师，是意图的表达者。

沉淀粒度：持久代码与瞬态代码的分化

Agent 没有消灭代码，而是让代码分化成了两种形态。

瞬态代码是 on the fly 生成的一次性中间产物，生命周期是分钟级，用完即弃。周报系统每次运行时生成的 Python 数据处理脚本就是这种——它根据当前的意图和数据源动态生成，执行完拿到结果，代码本身就不再有用。你不会把它提交到 Git，就像你不会把 JIT 编译器每次生成的机器码存进版本库。

沉淀代码是经过验证、反复出现、被证明有价值后固化下来的逻辑。周报系统里的 `connectors.yaml`（数据源定义）、CI 部署配置、`check_connectors.sh`（覆盖体检脚本）就属于这类——它们是经过多轮迭代沉淀下来的，仍然需要版本控制和测试。

这正是“意图驱动 + 代码沉淀”模式的体现：agent 快速生成瞬态代码来探索和响应意图，其中反复出现的、经过验证的模式被沉淀为持久化代码。两种代码共存，但对基础设施的要求完全不同。

镜像仓库建站是一个有趣的中间形态：agent 每次建站生成的配置代码不是用完即弃的——它们被提交到 Git、走 CR 流程、发版上线。但生成过程本身是 on the fly 的，agent 根据 SOP 意图和当前建站参数动态生成代码，人审批后固化。这里的人机协作分界线很有意思——**由风控机制决定，由出错代价决定**。生产环境的配置推送技术上完全可以自动化，但因为错推可能导致线上故障、缺乏灰度回滚能力，所以留给人做。

基础设施的使用者也在分化。Git、CI、部署平台仍然在运行，但它们越来越多地被 agent 调用而非被人直接操作。与此同时，一个反直觉的规律出现了：越是动态的系统，越需要坚固的静态约束。类型系统、沙箱、权限边界——这些确定性的约束成为 agent 系统可靠运行的护栏。

循环频率：从发布周期到运行时反馈

传统迭代周期大家都经历过：需求评审 → 开发（天/周）→ 测试 → 发布 → 下一个周期。每个环节都有等待时间，有审批流程，有排期协调。一个从意图到上线的完整循环，快的团队可能一两周，慢的团队可能一两个月。

Agent 参与后，这个周期可以压缩到分钟级。回到周报系统——我在一轮迭代里改了多次意图，每次改完 agent 立即重新生成代码并部署。不需要等 CR、不需要等 CI 排队、不需要等发布窗口。意图的表达、代码的生成、结果的验收在同一个会话里完成。

更准确地说，**迭代的性质变了**——从离线的、批量的、有审批流的变更，变成了在线的、增量的、实时反馈的演化。这个区别很重要。传统迭代是离散事件——每次发布是一个节点，两次发布之间系统是静止的。Agent 参与后，迭代变成了连续过程——系统在意图和反馈的驱动下持续演化，“发布”这个概念本身变得模糊了。

三个变量同时变化的效果是乘法而非加法。桥梁带宽提升让意图可以更快地进入系统，沉淀粒度变细让代码可以更灵活地生成和丢弃，循环频率跃迁让整个“意图 → 代码 → 反馈”的循环从离散事件变成了连续过程。这就是为什么原有 infra 会系统性失配——整个节奏变了，每个环节的设计假设都在同时失效。

03

把动态性推到极致

以上三个变量的讨论聚焦在 Developer Infra——agent 承担的是研发的桥梁角色。但如果沿着这个逻辑继续推，会触及一个更大的问题。

传统软件有一个根本假设：**意图可枚举**。产品经理写 PRD，本质是把用户的无限意图空间压缩成有限的 feature 集合。整条管线能工作的前提是用户的核心意图可预测、变化速度慢于发版速度。当意图的多样性和变化速度超过发版频率时，管线在架构上就不支持了。

这意味着 agent 在承担研发角色之外，还会承担产品经理的部分职能——在运行时直接理解用户意图并生成执行路径。产品经理从“意图的唯一入口”变成“意图的策展人”：长尾意图由 agent 实时响应，产品经理巡检后把反复验证的模式固化为代码。如果面对开放意图空间，agent 占比就有一个不可压缩的下界——那部分永远没法固化成代码，因为你不知道明天用户会提出什么。

这个话题再展开就进入 Product Infra 的范畴了，本文不深入。但它说明三个变量的变化不是孤立的技术现象，它会沿着“意图驱动”的逻辑一路推到产品形态和组织分工的层面。

频率变化是根因。根因确认之后，下一个问题是：到底哪些地方在失配？失配的深层原因是什么？在提出设计原则之前，先看三个实践案例。

04

从三个案例看 agent 与 infra 的摩擦

案例一：多角色 agent 研发系统

我们基于内部研发 CLI 搭建了一个多角色 agent 系统，在云服务器上运行多个 agent 角色——discovery（需求理解）、delivery（代码开发）、examiner（Code Review）等——覆盖从需求理解到发布上线的完整研发链路。每个角色通过研发 CLI 操作代码仓库、MR、CR、发布流水线、feedback。

这个系统里有一些东西做得不错。研发 CLI 本身已经相当 agent 友好——结构化 JSON 输出（agent 特别喜欢用 `--format json`，哪怕没要求也会主动加上）、统一的命令体系覆盖研发全流程。完整链路中，除了“去监控平台看日志和服务情况”这一步之外，其他环节通过 skill 文档和 `cli -h` 基本能让 agent 自己适应。公司基于统一鉴权框架统一了多个 CLI 的身份体系，也大幅降低了跨工具的身份管理成本。

但实际跑起来，痛点非常密集。

- **运行环境全靠手工搭建。** Agent 的运行环境不是被"产品化准备好"的——一台云服务器上同时维护多个模型入口、多套 CLI 工具、多个 agent profile、多套凭证和代理配置。想并行跑多个 agent 角色，权限容易串，环境容易乱。以 delivery 角色为例，它需要的"完整工作环境"包括：本次任务的 workspace 和目标 repo、前序环节（discovery / standards）已产出的 artifact 和上下文、本任务允许的工具和外部副作用范围、隔离的凭证和身份配置、对应的 skill 和 prompt、沟通和 handoff 通道。这远不是一个"沙箱"能覆盖的——它需要的是一个根据角色、任务、权限范围自动组装的完整工作环境。我们姑且把这个概念叫做 **harness**。
- **研发规范没有机器可读的稳定承载。** "这个应用到底应该怎么研发、测试、发布"——Dockerfile 怎么写、测试命令是什么、发布路径走日常→预发→正式还是项目环境→预发→正式、CR 和 pipeline 怎么创建——这些标准散在人的经验、历史任务、平台页面、仓库文档里。Agent 只能临场猜，或者靠 prompt 里硬编码。
- **接口还带有偏"人用"的痕迹。** MR 的全局 id 和项目 iid 缺少强类型化区分，agent 经常传错。某些隐性规则散落在团队口头知识里——比如"回复 MR 评论必须回复 root 评论，回复在子评论下面是看不到的"。 `--dry-run` 只停留在研发 CLI 层，无法穿透到底层平台。Pipeline 报错或超时后，agent 不知道下一步该干什么——状态机的边界对 agent 不显式。
- **验证的难度梯度极大。** CLI 验证很简单（输入输出确定性高），app 验证困难（涉及 UI 交互和状态），web 验证超级困难（浏览器环境、异步渲染、用户交互路径爆炸）。目前只能靠人为打 skill 补丁勉强解决 app 的验证。
- **Code Review 缺上下文。** 当 agent 产出的 MR 数量爆炸式增长，reviewer 如果不知道需求是什么、为什么这么改、哪些能力必须保留、这个 MR 在系统设计里承担什么角色，就只能看表面问题——代码风格、命名、是否有测试。真正危险的缺陷在更上层：需求理解错了、scope 漏了、方案方向错了、破坏了系统整体设计。Review 同样需要 harness——自动准备原始需求、discovery 产出的上下文和验收标准、研发规范、MR diff 加 CI 状态、本次变更影响的上下游。
- **没有可重复的实验场。** 很多验证只能拿真实仓库、真实 MR、真实 CI 来跑——生产任务变成了集成测试。需要一个可重复创建和关闭 MR、可控 CI pass/fail、可控 reviewer approve/reject 的实验场，或者一套贯通所有环节的 dry-run。

从这个案例中浮现出的问题覆盖了 agent 研发链路的几乎每个环节：运行环境、接口设计、规范承载、验证体系、上下文传递。这些不是孤立的功能缺失。

案例二：配置推送——agent 的自主程度取决于什么

镜像仓库建站案例中，一个同事不敢让 agent 推送配置。

他的顾虑很具体：配置推送的权限管控几乎没有——agent 逻辑上可以误推任何配置，包括其他高风险业务的配置。这是对 infra 安全能力缺失的精确识别，跟"不信任 AI"无关。

但当我问：如果配置推送做了严格资源归属治理和权限管控，并且提供了强 dry-run 能力——

```
1 $ config-tool publish data=xxx.json --dry-run
```

```
2 output:
3   diff: xxx
4   affected apps: xxxxx, yyyy, zzz
5   affected pods: about 1,024
```

他的回答是：如果是这样，就比较放心让 agent 自己推送配置了。

这个对话值得停下来细想。阻碍 agent 自主操作的瓶颈在哪里？不在 agent 侧——agent 的能力足够完成这个操作。瓶颈在 infra 侧——没有资源归属治理（配置属于哪个应用、哪个团队、哪个环境，没有机器可读的归属关系），没有 dry-run（推之前不知道会影响什么、影响多大），没有分级策略（影响 3 个 pod 和影响 1,024 个 pod 走同一条路径），回滚能力不足（推错了没有秒级回滚）。

Agent 能不能自主操作，不取决于 agent 有多聪明，而取决于 infra 提供了多强的安全护栏。

这个判断可以推广。所有生产环境的资源变更——应用发布、数据库 migration、网络策略、DNS 修改——每种变更类型在“资源归属 × 影响可预知 × 渐进信任 × 可回滚”四个维度上的安全保证强度不同，agent 的自主空间也就不同。回滚能力把不可逆操作变成可逆操作，等于降低了风险等级——这是 infra 能为 agent 做的最有杠杆的事之一。

案例三：身份与鉴权的碎片化

Agent 在使用基础设施时，最常遇到的阻塞往往是“身份不对”——比“不会用”更频繁。

公司基于统一鉴权框架统一了多个 CLI 的身份，但 git 仍用独立的 SSH 体系。对人来说配一次就行；对 agent 来说是两套凭证管理逻辑、两套轮转机制、两套报错模式——harness 初始化、错误排查、多角色配置的复杂度都翻倍。

沙盒环境的问题更尖锐。Agent 7×24 无人值守运行时，传统鉴权全部失效——Web SSO 需要浏览器（沙盒没有），短期 Token 2-8 小时过期（凌晨 3 点 token 过期，agent 直接卡死，需人工介入刷新）。解法是引入长效 Private Token——一次生成、长期有效、完全 headless、后端零侵入。这是一个正确的方向：“从以人为中心的短会话，向以 token 为中心的长效凭据演进。”

但即使有了统一鉴权和 Private Token，agent 仍然面对多套身份体系并存的局面——统一鉴权 for CLI tools、SSH for git、Private Token for 代码平台 API。每一套需要单独配置、单独排错。

为什么身份碎片化在人的时代不是大问题，到 agent 时代突然变成了瓶颈？

结构性原因是：**角色分工曾经屏蔽了身份碎片化的复杂度**。原来人分为前端、后端、运维、设计、数据等角色，各系统为各自角色设计，各自有各自的身份体系。但每个人实际只接触自己角色对应的 2-3 套系统——人的角色分工是一种天然的关注点分离，把身份碎片化的复杂度屏蔽在了各自的工作边界内。前端不需要碰运维的身份体系，运维不需要碰数据平台的凭证。

Agent 打破了角色边界。一个 delivery agent 要 git clone (SSH)、研发 CLI 提 MR (统一鉴权)、操作中间件配置 (中间件身份)、查监控 (监控平台身份)——它一个“角色”横跨了原来多个人类角色的身份体系。身份体系数量从“每人 2-3 套”膨胀到“每 agent 5-8 套”。

然后第二层放大效应是频率。人一天切换身份体系的次数可能就几次，每次切换的摩擦被工作间隙吸收。Agent 一分钟内可能连续调 git → 研发 CLI → 中间件 → 监控，摩擦被放大了成了瓶颈。**两个乘法因子：角色融合导致身份体系数量膨胀 × 工作频率导致切换次数膨胀。**这也呼应了“三个变量乘法效应”的逻辑——桥梁从人换成 agent 之后，人的角色分工所屏蔽的复杂度全部暴露给了 agent。

理想状态是 agent 拿到一个身份，这个身份在所有 infra 上都能用。

三个案例的共性

三个案例的具体问题各不相同——运行环境、接口设计、资源变更安全、身份鉴权——但根源是同一个：**这些 infra 都是为人设计的。**

设计假设是：操作者有常识（不会手贱推别人业务的配置）、有责任心（出错后能人工修复）、操作频率低（一天切换几次身份不是问题）、能从口口相传的隐性知识中填补系统概念的裂缝（“回复 MR 评论必须回复 root 评论”）。

Agent 不符合这些假设中的任何一条。它无常识（会 hallucinate）、操作频率极高、出错后需要自动恢复、面对概念裂缝会卡住或猜错还不自知。

那面向 agent 设计基础设施，和面向人设计，原则上有什么不同？

05

设计原则：

从 People-Oriented 到 Agent-Oriented

从案例中浮现出一个根本性的判断：**Agent 的自主程度是 infra 安全能力的函数。**

传统 infra 的设计是 People-Oriented 的：安全靠人的自我约束加事后审计。你不会手贱去推别人业务的配置——因为你知道不该这么做，尽管系统并没有禁止。系统靠人的自我约束来弥补安全机制的缺失。

面向 agent 的 infra 设计是 Agent-Oriented 的：安全靠机制化保证——资源归属、权限管控、dry-run、分级策略、自动回滚。

从 People-Oriented 到 Agent-Oriented，本质上是把安全模型从“依赖人的自我约束”重建为“依赖 infra 的机制保证”。给 infra 补能力，而非给 agent 加限制。Infra 的安全能力越强，agent 的自主空间越大。

这和前面的核心判断形成对称：“人容忍概念的裂缝，agent 需要概念的闭合”——认知层面。“人容忍安全机制的缺失（靠常识弥补），agent 需要安全机制的闭合（靠 infra 保证）”——安全层面。两者共同指向同一个方向：**面向 agent 的 infra 设计，就是把原来隐式依赖人的东西——无论是认知、常识还是责任心——显式化为系统机制。**

具体来说，infra 需要在四个层次上提供能力，每一层都是在扩大 agent 的安全自主空间。

- **可理解 (Comprehensible) ——agent 能建立正确的心智模型。** 这是所有后续层的前提。系统的概念体系必须自治、完整、不依赖口口相传的隐性知识。回看案例一：研发 CLI 的 MR 评论规则、研发规范、某些字段含义——这些散落在人际网络中的隐性知识，对 agent 来说是概念的裂缝。但一个概念自包含的 CLI，即使有 200 个子命令、每个 30 个参数，agent 能通过多轮试探快速学会。**复杂度不是问题，不一致才是。** 运行环境也要自描述——用 manifest / devcontainer 声明依赖、输入、输出、允许工具，这是 harness 初始化的前提。
- **可操作 (Operable) ——agent 能安全可靠地行动。** 理解之后是行动。关键能力包括：可试探 (dry-run / preview、幂等重试——案例二中有 dry-run 同事就“比较放心了”)、操作原子组合 (输入输出类型化——案例一中 MR id 和 iid 混用就是反例)、隔离执行 (每个任务一个 sandbox，试错代价可控)、凭证不进 sandbox (通过 vault / egress proxy 注入能力，agent 不持有明文凭证)、渐进信任 (低风险 agent 自主执行，高风险 agent 准备好变更计划和 dry-run 结果、人来点按钮——回滚能力把不可逆操作变成可逆操作，等于降低风险等级)。
- **可感知 (Observable) ——infra 把状态和结果清晰地交回 agent。** Agent 行动之后需要感知结果。沉默和含糊是 agent 的敌人——Unix 的“Silence is golden”对 agent 有害。状态必须 API 可查、结构化、实时 (案例一中 MR 的 reviewer、approval、readyToMerge、CI、discussion 是不同 gate，不能靠一段文本判断)。每次响应——成功或失败——都要提供足够的语义信息让 agent 决定下一步 (pipeline 报错后 agent 不知道下一步该干什么，就是反馈语义不足)。结果要可判定——把 agent 的产出转成 CI 能识别的 pass/fail，必要时保留人工 review gate。
- **可追溯 (Traceable) ——过程不丢失、可恢复、可回放。** Agent 的工作可能跑几分钟也可能跑几小时。状态可恢复 (snapshot / checkpoint，避免长任务因容器故障从零开始)、过程可回放 (至少保存 artifact，更进一步保存 execution trace，可以 fork/replay)。这里还有一个容易被忽略的维度：**通过 agent 的行为来观测 infra 本身的设计质量。** Agent 反复重试某个 API → 反馈语义不足；调用顺序混乱 → 前置条件没有显式化；用错身份 → 身份体系碎片化。人遇到这些问题会默默绕过去 (问同事、手动修、记住坑)，不产生可观测信号。Agent 每次都会忠实地撞上去，频率极高——agent 的失败模式是 infra 设计缺陷的放大器。这让 infra 的设计质量从主观判断变成了可度量的东西。

四层之间有递进关系：**先理解** (建立心智模型) → **再操作** (安全、隔离、可试探) → **感知反馈** (结构化、语义丰富、可判定) → **全程可追溯** (可恢复、可回放、可度量)。每一层同时覆盖 API 接口设计和运行时环境两个维度。

06

行业趋势与我们可以做什么

我们从内部案例中提炼的判断，并不是孤立的。行业里多家公司从不同的切入点，独立得出了高度一致的结论。

行业在做什么

并行 agent 需要并行 infra。 Superset 是一个面向 multi-agent 开发的 IDE，每个开发者同时跑最多 12 个 coding agent，三人团队每天产生约 600 个 preview deployment，平均构建时间约 30 秒 [1]。他们的关键洞察：当开发者同时运行多个 agent 时，底层 infra 的任何一层如果变慢，上层的并行性就会崩塌——十二个工作流压缩成一个队列，本该几分钟完成的任务变成几小时。

这验证了两件事。一是 harness 的供给必须是即时的——“环境初始化需要 5 分钟”在 12 个并行 agent 面前就是 60 分钟的等待。二是 agent 数量对 infra 的压力是乘法关系：1 个开发者 → 12 个 agent → 12 套隔离环境 → 12 条并行 pipeline → 12 个即时部署。Infra 的吞吐量必须线性 scale with agent 数量，否则并行性在 infra 层被压缩回串行。三人团队没有 platform engineering，整个 infra 靠平台的即时供给能力支撑——这暗示了一个方向：面向 agent 的 infra 不应该需要专门的团队来维护环境管理。

Human DX 和 Agent DX 是正交的。 Google 的 Workspace CLI (gws) 从第一天就按 agent-first 设计 [2]。作者提出一个精确的区分：Human DX 优化的是可发现性 (discoverability)，Agent DX 优化的是可预测性 (predictability)。

关键实践包括：schema 自省替代静态文档 (`gws schema drive.files.list` 输出完整方法签名，CLI 成为 API 运行时状态的权威来源)、输入加固防 hallucination (路径沙箱化、拒绝控制字符、资源 ID 校验——因为 agent 会 hallucinate 出 `../../.ssh` 这样的路径穿越)、发布 SKILL.md 文件编码 agent 无法从 `--help` 推断的不变量 (“写操作必须先 dry-run”、“list 调用必须加 `--fields`”)、

响应消毒防 prompt injection。这些实践和我们在"可理解"（概念自包含）、"可操作"（可试探）层的原则高度吻合。

这里面有一个观点我认为很关键——**agent 不是可信的操作者 (The agent is not a trusted operator)**。它会 hallucinate，会生成 `../../../../ssh` 这样的路径穿越，会在资源 ID 里嵌入查询参数。面向 agent 的设计需要把 Unix 哲学里"Trust the user"的假设翻转过来。

Credential Brokering 的行业趋同。 多家公司独立得出了同一个结论：agent 需要凭证来访问外部服务，但 agent 本身不能被信任持有这些凭证——因为 prompt injection 可能导致凭证泄露 [3]。

解法是在 agent 和凭证之间画一条信任边界。引入一个 credential broker（代理层）：agent 发出请求时携带占位符（如 `__github_token__`）而非真实凭证，broker 认证 agent 身份后将占位符替换为真实凭证转发请求，agent 全程未见到真实凭证。Anthropic（Managed Agent Infrastructure）、Vercel（Sandbox 凭证注入）、Cloudflare（Outbound Workers）、LangChain（Sandbox Auth Proxy）各自在不同层实现了同一范式。

回看案例三中的身份演进路径——短会话 token → 长效 Private Token → credential brokering——这是一条从"agent 持有凭证"到"agent 完全不接触凭证"的渐进线。多家公司独立趋同到同一范式，说明这是 agent 安全模型的必然走向。

我们可以做什么

结合内部案例的痛点和行业的方向，以下是几个高杠杆的行动方向。

- **Harness 平台化。** 把 agent 运行环境从"每个团队自己在云服务器上搭"变成平台的内建能力。Harness spec 声明式定义——角色、工具、凭证、workspace、skill——handoff 时自动初始化，完成后自动回收。核心约束是即时供给：并行 agent 数量对环境供给是乘法压力，初始化时间直接决定并行效率。本质上还是云原生 / serverless 的思路，但组装的对象从"应用"变成了"agent 工作环境"。
- **统一身份体系。** Agent-native 的身份——不是借用人的 token，而是 agent 拥有自己的机器人身份。这个身份在所有 infra 上通用（消除统一鉴权 vs SSH 的碎片化），自动轮转、最小权限、操作可追溯到具体的 agent 实例和意图。身份与 harness 联动：harness 初始化时注入角色对应的凭证，不同角色天然隔离。远期方向是 credential brokering——agent 能使用凭证但不持有凭证。
- **Dry-run 基础设施。** 在基础设施层提供统一的"变更预览"能力，而非每个工具各自实现。资源归属治理（这个配置属于哪个应用、哪个团队、哪个环境）是 dry-run 的前提——没有归属关系就没法计算影响范围。Dry-run 的输出应该标准化：diff + 影响范围 + 影响规模 + 风险等级 + 是否可回滚。这是渐进信任机制的基础——根据 dry-run 输出的风险等级，决定 agent 自主执行还是交给审批。配置推送案例中同事的判断逻辑——"有 dry-run 就放心"——应该被泛化为一种 infra 能力，覆盖所有生产环境资源变更。
- **验证体系的演进。** 传统软件测试和 agent 评测正在从两端走向融合。传统测试验证"代码是否正确实现了意图"，agent 评测验证"模型是否正确理解了意图"——当系统是 agent + code 的混合体，两者融合成同一件事：从意图到结果的端到端验证。

- 融合后的形态有几个特征：断言方式从精确匹配（`output == expected`）演进到约束满足（输出满足一组约束——类型正确、不违反安全策略、业务逻辑自洽——具体实现路径可以不同）；确定性验证与概率性验证分层共存（同一个测试中，确定性部分用传统断言，概率性部分用 LLM-as-judge 或统计方法）；测试从独立阶段变成持续验证——CI 管固化代码（低频、确定性），嵌入式验证运行时管 on the fly 代码（高频、概率性、约束验证）。
- 一个判断：**验证基础设施的投资优先级应该高于生成能力**。生成能力的提升是模型厂商在推的事，验证能力的提升是 infra 团队该做的事——而验证的可靠性直接决定了 agent 的自主空间。
- Agent 行为驱动的 Infra 质量度量**。这是一个容易被忽略但杠杆很高的方向。通过 agent 的行为模式——重试频率、错误类型、绕路路径——来度量 infra 的 agent 友好程度。Agent 反复重试某个 API，说明反馈语义不足；调用顺序混乱，说明前置条件没有显式化；频繁在两个系统之间做格式转换，说明接口不可组合。人遇到这些问题会默默绕过去，不产生可观测信号。Agent 每次都会忠实地撞上去，频率极高——agent 成为 infra 设计的持续压测者。用得越多，暴露的设计缺陷越多，改进方向越清晰。

今天的 Git、CI/CD、Code Review 系统不会消失，但它们的管辖范围会收缩到“持久化代码”的领域——而那个领域的面积在缩小。真正需要被建设的，是支撑 agent + code 动态混合体的新一代基础设施。方向已经清晰：给 infra 补能力。Infra 的能力边界，就是 agent 的自主边界。

致谢

感谢残风、星楚、晚甜提供的实践案例和深入讨论，感谢飞桨分享的沙盒环境 Agent 鉴权实践 2。本文的很多判断来自这些一线实践的碰撞。

参考资料

References

- [1] Vercel Blog, "How Superset built the IDE for AI agents on Vercel", 2026-05-10. <https://vercel.com/blog/how-superset-built-the-ide-for-ai-agents-on-vercel>
- [2] Justin Poehnelt, "You Need to Rewrite Your CLI for AI Agents", 2026-03-04. <https://justin.poehnelt.com/posts/rewrite-your-cli-for-ai-agents/>
- [3] Tony Dang, "Credential Brokering for AI Agents", Infisical Blog, 2026-05-23. <https://infisical.com/blog/credential-brokering-for-ai-agents>

Footnotes

- 文中“研发 CLI”指公司内部覆盖代码仓库、MR、CR、发布流水线等研发全流程的命令行工具。“统一鉴权框架”指公司内部统一多个 CLI 身份体系的鉴权框架。两者均为内部系统，名称已做脱敏处理。↩
- 飞桨，《沙盒环境 Agent 7×24 小时工作，免登鉴权怎么破？》，内部技术社区，2026-05-26。↩

*封面头图来自MoMa，侵权删。

欢迎留言一起参与讨论~

